TRACK 2 - AGENTIC AI

Radeon Requirement Guardian

A fully local document-to-action agent with planning, tools, memory, execution, and independent verification.

AMD Radeon PRO W7900D | ROCm 7.2.1 | vLLM | Qwen3

1. Application Scenario and Value

Organizations regularly receive dense requirement documents: procurement requests, compliance policies, audit procedures, grant rules, and project submission instructions. Missing one mandatory item can invalidate an otherwise strong deliverable. Cloud assistants are often unsuitable because the documents can be confidential.

Radeon Requirement Guardian runs the complete reasoning workflow locally. It converts a source document and user goal into an auditable action report with evidence citations, checkboxes, risks, and a separate verification result. It produces a deliverable rather than a generic chat response.

Primary user

Project owners, compliance teams, procurement staff, and developers handling private documents.

Input

A PDF or text requirement document plus a concrete execution goal.

Output

Evidence-backed checklist, execution plan, risks, missing information, verification JSON, and trace.

Privacy

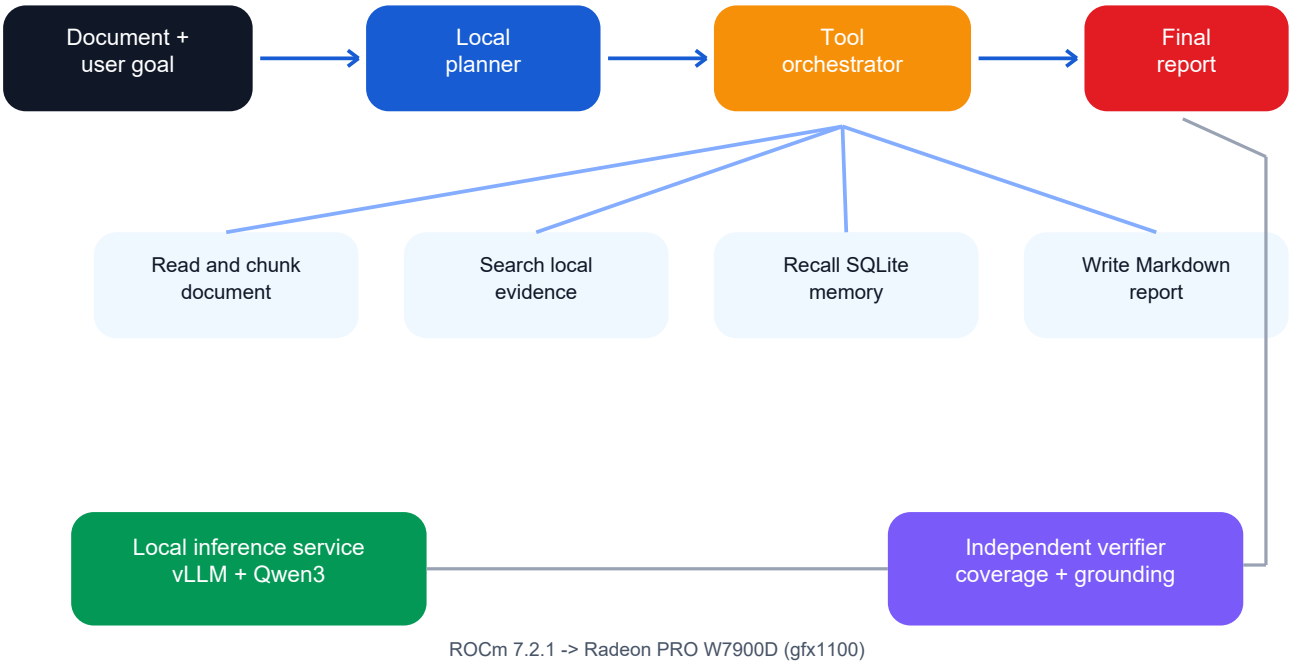
The document, model inference, memory database, and generated report remain in the local workspace.

Hackathon demonstration

The included sample asks the agent to audit hackathon submission rules. The same workflow transfers directly to enterprise checklists and private project delivery.

2. Agent Architecture

The system uses a deterministic orchestration loop around three local model stages. Tool outputs are explicit and every factual requirement in the final report must point back to a retrieved evidence chunk.



Execution sequence

1. Read and chunk the uploaded document.
2. Recall relevant outcomes from the local SQLite memory.
3. Ask the local planner for a goal, steps, and document search queries.
4. Execute lexical evidence searches and de-duplicate the retrieved chunks.
5. Generate a grounded Markdown checklist from the plan, evidence, and memory.
6. Run an independent verifier for coverage and unsupported claims.
7. Write the report locally and store a concise memory for later tasks.

3. Core Capabilities

Capability	Implementation
Reasoning and planning	Converts the task into an executable plan and targeted evidence queries.
Tool use	Reads files, searches local chunks, recalls memory, and writes a report.
Memory management	Stores concise outcomes in SQLite and recalls related prior work.
Task execution	Creates a persistent checklist file instead of only returning conversation text.
Verification	A separate model call checks coverage and unsupported claims.
Auditable experience	UI tabs expose the report, verifier result, complete trace, and performance data.

4. Model and Local Deployment

The submission targets Qwen3-0.6B in FP16, served through vLLM's OpenAI-compatible localhost endpoint. The small model makes the demo fast and reproducible while keeping all reasoning on Radeon. No API key or remote model service is required.

Component	Target
GPU	AMD Radeon PRO W7900D, RDNA 3, gfx1100, 48 GB VRAM
Runtime	ROCm 7.2.1 with a compatible vLLM build
Model	Qwen/Qwen3-0.6B, FP16
Application	Python, Gradio, Requests, pypdf, SQLite

5. AMD Radeon Optimization

The optimization strategy focuses on a measurable application baseline that can be reproduced during judging.

- FP16 weights reduce memory footprint and memory traffic relative to FP32.

- The model remains loaded in Radeon VRAM behind one persistent vLLM service.

- Planner, writer, and verifier share the same server process instead of reloading the model.

- vLLM enables continuous batching and efficient request scheduling on the ROCm backend.

- The workshop-safe baseline uses eager execution; default compiled execution can be measured as a follow-up comparison.

Measured evidence

Each run reports local model-call count, token usage returned by vLLM, aggregate inference time, end-to-end time, and effective output throughput. The demo also captures live `rocm-smi` output and the localhost model endpoint. Effective throughput is explicitly labeled as an application-level metric, not raw kernel decode throughput.

Metric	Collection method	Purpose
GPU and VRAM	rocm-smi	Prove Radeon execution
Inference time	Timer around each localhost request	Measure local reasoning latency
Effective output throughput	Completion tokens / inference seconds	Compare identical agent runs
End-to-end time	Timer around the full agent workflow	Represent user experience
Coverage score	Independent verifier	Track functional quality

6. Demonstration and Evaluation

The three-minute demo shows the Radeon device, local vLLM endpoint, one complete sample task, the evidence-backed report, verifier JSON, tool trace, persistent memory event, and measured metrics. Offline unit tests validate chunking, retrieval, JSON extraction, and SQLite memory.

Success criteria

- The report identifies all explicit sample requirements without inventing new ones.

- Every factual requirement references a local evidence chunk.

- The trace contains planning, retrieval, report writing, verification, and memory events.

- No cloud AI API or API key is used.

- The demo includes real measurements from the target Radeon workspace.

References

AMD ROCm vLLM documentation: https://rocm.docs.amd.com/projects/ai-ecosystem/en/latest/inference/vllm.html
Source code and reproducible instructions are included with this submission.